

Brief: Surf Security Integration — Secure Browser Partnership

version v0.4.11

date 19 Feb 2026

from Human (project lead)

to Ambassador (lead), Architect, AppSec

type Partnership research brief — integration concept

The Idea

Surf Security provides a managed secure browser for enterprises. SG/Send provides zero-knowledge encrypted file and message transfer in the browser. The integration concept: **what if SG/Send running inside a secure browser could offer security guarantees that are impossible in a normal browser?**

This brief captures the ideas to explore. It is not a partnership proposal yet — it's the research that would inform one.

Note: while this brief uses Surf Security as the specific example, most of these ideas apply to any enterprise secure browser (Island, Talon/Palo Alto, etc.). Surf is the starting point because of existing relationships.

What a Secure Browser Gives Us

A normal browser is an untrusted environment. We can't verify:

- Whether extensions are reading the page
- Whether the network is intercepting traffic
- Whether the device is compromised
- Whether screenshots are being taken

A secure browser like Surf controls all of this. It's a managed environment where the browser vendor (and the enterprise IT team) can enforce policies. This changes the security model fundamentally.

Capabilities We Could Leverage

Capability	What It Enables for SG/Send
No malicious extensions	Guarantee that no extension is reading decrypted content from the DOM
DLP (Data Loss Prevention)	Prevent copy-paste of decrypted content outside the browser
Screenshot prevention	Prevent screen capture of decrypted content
Network integrity	Guarantee no TLS interception / MITM proxy between browser and server
Device posture	Know whether the device meets security requirements before allowing decryption
Session management	Know that the browser session is authenticated and not hijacked

Capability	What It Enables for SG/Send
Controlled environment	The browser itself is the security boundary — not just the web app

Integration Ideas

1. Browser-Managed Key Storage

Problem today: encryption keys are in localStorage (one of the P3 findings from the security review). Any script with page access can read them.

With a secure browser: the browser itself could manage key storage — using a secure enclave, a browser-level keychain, or a protected storage API that's not accessible to page scripts or extensions.

Research question: does Surf (or any secure browser) expose a protected storage API that web apps can use? Something more secure than localStorage but accessible to our JavaScript?

2. Public/Private Key Management

The bigger version of #1: the browser manages the user's public/private key pair. SG/Send doesn't need to build its own key management — the browser provides it. Keys never leave the browser's secure storage. PKI becomes a browser-level feature, not an application-level feature.

Research question: can we access the browser's credential management or certificate store from a web application? Do secure browsers expose APIs for this?

3. Links That Only Work Inside the Secure Browser

The idea: generate SG/Send links that include a requirement: "this content can only be decrypted inside a Surf browser." If someone opens the link in Chrome or Firefox, they see a message: "This content requires a secure browser environment."

How it could work: the SG/Send page detects whether it's running inside a Surf browser (via a browser-specific API, user agent, or challenge-response). If yes, proceed with decryption. If no, refuse.

Use case: a bank sends a confidential document to a client. The document can only be viewed inside the bank's managed browser environment. The client can't forward the link to someone using a normal browser.

4. Compromised Device / Network / Session Detection

The idea: before decryption, the SG/Send page asks the browser: "Is this device, network, and session secure?" The browser responds with a security posture assessment. If the posture is below a threshold, decryption is blocked.

Scenarios:

- Device failed a compliance check → "Cannot decrypt: your device does not meet security requirements"
- Network is untrusted (public WiFi, known proxy) → "Cannot decrypt: your network is not secure"
- Browser session is not authenticated → "Cannot decrypt: please log in to your secure browser"

This is the "browser security checklist" idea from the security review brief — but implemented by the browser itself, not heuristically by the web app.

5. Self-Expiring Messages with Enforcement

Today: we can set a link to expire after a time period. But the recipient can download the content before expiry and keep it forever.

With a secure browser: the browser could enforce that decrypted content is never saved to disk. The content exists only in the browser tab. Close the tab, the content is gone. The browser prevents file saving, copy-paste, and screenshots. This is true "view once" — the content is consumed and then destroyed.

Research question: does Surf support policies that prevent file downloads, copy-paste, and screenshots for specific web applications?

6. Secure Copy-Paste Between SG/Send Instances

The idea: within a Surf browser, copy-paste between SG/Send tabs could be allowed (the content stays within the secure environment), but copy-paste to external applications is blocked.

This enables workflows like: decrypt a document in SG/Send → copy relevant text → paste into another SG/Send message → re-encrypt and send. All within the secure boundary.

The Partnership Angle

Why This Is Good for Surf's Customers

Surf's customers are security-conscious enterprises. They've already invested in a secure browser. SG/Send gives them a secure file and message transfer capability that leverages their browser investment — features that only work because they have a managed browser.

This is a differentiation story for Surf too: "Use our browser, and your SG/Send transfers are MORE secure."

Why This Is Good for Us

- **Channel:** Surf's customer base is our target market
- **Credibility:** integration with an enterprise security vendor validates our product
- **Features:** capabilities that are impossible in a normal browser (DLP, screenshot prevention, secure key storage) become possible
- **Revenue:** joint go-to-market, co-branding, professional services for deployment

Customer Overlap

Surf's customers are already security-conscious. They've already decided to invest in browser security. This is the exact audience that values zero-knowledge encryption, two-channel security, and the privacy features we offer. The overlap is strong.

Research Deliverables

Task	Owner	Priority
Research Surf Security's API / extension capabilities — what can a web app detect and use?	Architect, Developer	P3
Research other secure browsers (Island, Talon/Palo Alto) — are APIs similar?	Architect	P4
Assess: which of the 6 integration ideas are technically feasible today?	Architect, AppSec	P3
Research DLP capabilities — can Surf prevent copy-paste and screenshots for specific web apps?	AppSec	P3
Research browser credential/key storage APIs — can we use the browser's secure storage?	Developer, Architect	P3
Draft a one-page partnership concept to share with Surf if research is positive	Ambassador	P4
Connect this to the Chrome extension detection research from the security review brief — the secure browser is the "solved" version of that problem	AppSec	P3

Connection to Previous Work

The security review brief (v0.4.7) included a research task: "Can we detect if Chrome extensions have permission to read the current page?" and the vision of a browser security checklist (🟢🟡🔴 indicators).

A secure browser integration is the enterprise answer to that question. Instead of heuristically guessing whether the browser is secure, we can *know* — because the browser tells us.

For non-enterprise users on normal browsers, we still pursue the heuristic detection approach. For enterprise users on secure browsers, we get definitive answers. Both paths are valuable.

This document is released under the Creative Commons Attribution 4.0 International licence (CC BY 4.0). You are free to share and adapt this material for any purpose, including commercially, as long as you give appropriate credit.